

Volet D –

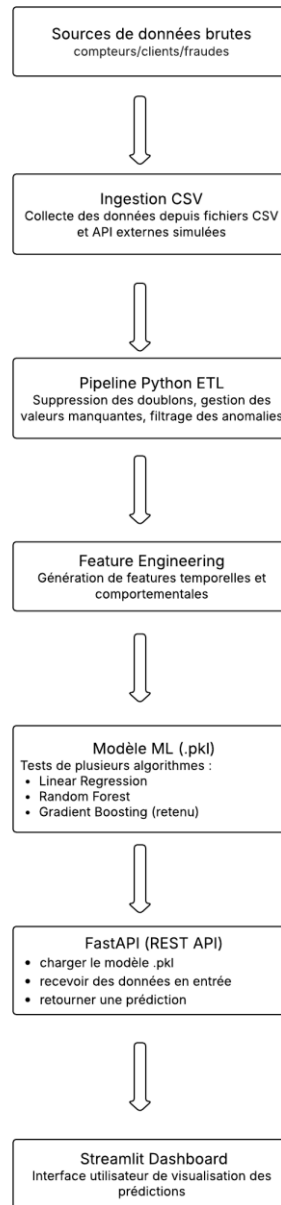
Ingénierie Logicielle et Data Engineering

1. Architecture technique de la plateforme

L'objectif de l'architecture Néovolt est de fournir une chaîne complète allant de l'acquisition des relevés de consommation jusqu'à la restitution des résultats aux utilisateurs finaux.

L'architecture retenue est organisée selon une approche modulaire afin de faciliter la maintenance, l'évolution et le déploiement.

- Flux global :



- Cette séparation permet d'isoler clairement les responsabilités de chaque composant.

2. Pipeline d'ingestion des données

❖ Objectif

Le pipeline a pour rôle de collecter, préparer et transformer les données nécessaires à l'entraînement et à l'exploitation du modèle de détection de fraude.

❖ Approche retenue

Dans le cadre du prototype, une architecture batch légère a été mise en œuvre à partir de fichiers CSV.

Cette approche constitue un équivalent simplifié des architectures industrielles reposant sur Kafka, Spark ou Airflow.

Le pipeline est organisé en plusieurs modules Python :

○ **Ingestion.py**

Responsable du chargement des différentes sources de données :

- relevés de consommation.
- Compteurs.
- Clients.
- fraudes confirmées.

○ **preprocessing.py**

Responsable :

- Du nettoyage.
- De la gestion des valeurs manquantes.
- De la fusion des jeux de données.
- Du feature engineering.

○ **training.py**

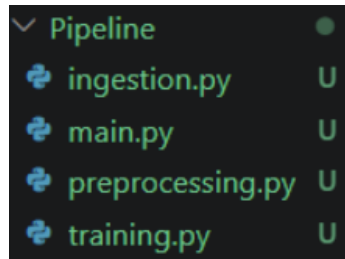
Responsable :

- De l'équilibrage du dataset.
- De l'entraînement.
- De l'évaluation.
- De la sélection du meilleur modèle.

○ Main.py

Orchestre l'ensemble du pipeline et permet sa réexécution complète à partir d'une simple commande.

Cette organisation s'inspire du fonctionnement des orchestrateurs de workflow tels qu'Airflow.



3. Stockage des artefacts

Le modèle final est stocké sous forme sérialisée grâce à Joblib.

- Fichier généré : modele_fraude.pkl

Cette approche permet :

- La réutilisation du modèle sans réentraînement.
- Le déploiement rapide.
- La séparation entre entraînement et inférence.

```
# =====  
# Save model  
# =====  
joblib.dump(best_model, "modele_fraude.pkl")  
  
print("Modèle sauvegardé ✓")  
  
return best_model, results
```

4. Exposition des services

- API REST FastAPI

Une API REST a été développée afin d'exposer le modèle aux autres composants du système.

- Endpoint principal :

POST /predict

Les données envoyées sont converties en DataFrame puis transmises au modèle.

L'API retourne :

- La prédiction.
- La probabilité estimée de fraude.

Avantages

- Découplage entre modèle et interface.
- Interopérabilité.
- Facilité d'intégration avec d'autres applications.

API Détection de Fraude Neovolt ^{1.0} ^{QAS 3.1}

default

GET

/ Home

POST

/predict Predict

5. Interface utilisateur

Une interface web a été développée avec Streamlit.

Elle permet :

- La saisie des caractéristiques du client.
- L'envoi des données vers l'API.
- L'affichage du niveau de risque.
- La visualisation de la probabilité de fraude.

Neovolt - Détection de Fraude Électrique

Analyse intelligente des consommations pour détecter les comportements suspects.



Données du client

Consommation (kWh)

119,99

-

+

Jour de semaine

0

▼

Conso J-1

110,00

-

+

Mois

1

▼

Moyenne 7 jours

100,00

-

+

Weekend

0

▼

Écart-type 7 jours

15,00

-

+

Surface (m²)

50,01

-

+

Variation (%)	Nombre de personnes
0,18 - +	6 - +
Conso / m ²	Puissance (kVA)
2,50 - +	6,01 - +

 Analyser le risque

Résultat de l'analyse

Probabilité fraude

14.47%

 CLIENT NORMAL

Analyse

Comportement normal

6. Déploiement Cloud

La solution a été déployée sur la plateforme Render.

- Deux services indépendants ont été créés :

- **Service API** : Héberge le backend FastAPI.
- **Service Frontend** : Héberge l'application Streamlit.

June 4, 2026 at 12:20 AM

✓ Live

@d1b638
interface modification

All logs
▼

Search

Jun 4, 12

Jun 4

```

12:44:27 AM [lw8sf] ==> Running 'uvicorn fast:app --host 0.0.0.0 --port $PORT'
12:45:01 AM ==> No open ports detected, continuing to scan...
12:45:01 AM ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
12:45:03 AM [lw8sf] INFO: Started server process [58]
12:45:03 AM [lw8sf] INFO: Waiting for application startup.
12:45:03 AM [lw8sf] INFO: Application startup complete.
12:45:03 AM [lw8sf] INFO: Uvicorn running on http://0.0.0.0:10000 (Press CTRL+C to quit)
12:45:03 AM [lw8sf] INFO: 127.0.0.1:60652 - "HEAD / HTTP/1.1" 405 Method Not Allowed
12:45:10 AM ==> Your service is live 🎉
12:45:10 AM [lw8sf] INFO: 35.227.183.65:0 - "GET / HTTP/1.1" 200 OK
12:45:11 AM ==>
12:45:11 AM ==> //////////////////////////////////////
12:45:11 AM ==>
12:45:11 AM ==> Available at your primary URL https://projet-neovolt-5.onrender.com
12:45:11 AM ==>
12:45:11 AM ==> //////////////////////////////////////

```

➤ liens:

- Fast api : <https://projet-neovolt-5.onrender.com>
- Interface : <https://projet-neovolt-3.onrender.com>

7. Scalabilité et résilience

Même si le projet est présenté sous forme de prototype, plusieurs principes de conception favorisent la montée en charge :

- Séparation des responsabilités.
- Découplage API / Interface.
- stockage indépendant du modèle ;
- déploiement cloud.

Dans une architecture industrielle, le pipeline pourrait évoluer vers :

- Kafka pour l'ingestion temps réel ;
- Spark pour les traitements distribués ;
- Airflow pour l'orchestration ;
- PostgreSQL pour le stockage ;
- Kubernetes pour le déploiement.

8. Documentation et reproductibilité

Le projet est entièrement reproductible.

Les étapes de relance sont :

1. Exécuter le pipeline d'entraînement.
2. Générer le fichier `modele_fraude.pkl`.
3. Lancer l'API FastAPI.
4. Lancer l'application Streamlit.

L'ensemble des composants est versionné et documenté.

9. Intégration globale

Le projet constitue une chaîne complète intégrant :

- ingestion de données ;
- préparation ;
- apprentissage automatique ;
- exposition de services ;
- interface utilisateur ;
- déploiement cloud.

Cette intégration répond aux objectifs du projet Néovolt en démontrant la capacité à concevoir une solution complète de détection de fraude énergétique, depuis la donnée brute jusqu'à l'exploitation opérationnelle des résultats.